PWR Racing Team - Rozwiązanie

Kacper Szmigielski

2026-10-13

Wstęp

Urzyte narzędzia

Zadanie 1

1.1

Algorytmy W tym zadaniu zdecydowałem się na przedstawienie paru różnych podejść ze względu na ich specyfikę.

Algorytm Zachłanny Dla każdego punktu iteruje po wszystkich innych punktach i jeżeli odległość od innego punktu jest mniejsza od threshold, to uśredniamy te punkty tworząc nowy punkt i dodajemy go do zbioru punktów. Robimy tak w pętli tak długo, aż nie zostaną już żadne punkty do połączenia.

Zalety:

- Intuicyjność i łatwość implementacji.

Wady:

- Wysoka złożoność obliczeniowa $O(n^2)$ w iteracjach.
- Wrażliwość na kolejność danych wejściowych.
- Efekt łańcuchowy (tendencja do łączenia osobnych grup przez pojedyncze punkty pośrednie).
- Brak mechanizmu wykrywania i odrzucania szumu.

Implementacja

```cpp
#include "Greedy.h" //Interfejs
#include <iostream>
#include <cmath>

static double calculateDistance(const Point& p1, const Point& p2) {
    return (std::pow(p1.x - p2.x, 2) +
            std::pow(p1.y - p2.y, 2) +
            std::pow(p1.z - p2.z, 2));
}

std::vector<Point> GreedyClusterer::run(const std::vector<Point>& points, double threshold) {
    std::vector<Point> currentPoints = points;
    bool mergedAny = true;

    // Iterujemy dopóki w poprzednim kroku udało się coś połączyć
    while (mergedAny) {

        mergedAny = false;
        std::vector<Point> nextPoints;
        std::vector<bool> merged(currentPoints.size(),  false);

        for (size_t i = 0; i < currentPoints.size(); ++i) {
            if (merged[i]) continue;

            Point current = currentPoints[i];

            for (size_t j = i + 1; j < currentPoints.size(); ++j) {
                if (!merged[j]) {
                    double dist = calculateDistance(current, currentPoints[j]);
                    if (std::pow(dist, 2) <= threshold) {
                        // Łączymy punkty (uśredniamy pozycję)
                        current.x = (current.x + currentPoints[j].x) / 2.0;
                        current.y = (current.y + currentPoints[j].y) / 2.0;
                        current.z = (current.z + currentPoints[j].z) / 2.0;
                        merged[j] = true;
                        mergedAny = true;
                    }
                }
            }
            nextPoints.push_back(current);
        }
        currentPoints = nextPoints;
    }

    std::cout << "Algorytm Greedy (iteracyjny) zakończony. Liczba punktów przed: "
              << points.size() << ", po redukcji: " << currentPoints.size() << "\n";

    return currentPoints;
}
```

Podczas projektowania rozwiązania rozważono również zastosowanie algorytmu DBSCAN oraz hierarchicznego grupowania danych. DBSCAN jest metodą grupowania opartą na gęstości, umożliwiającą identyfikację skupisk oraz punktów odstających. W rozpatrywanym problemie nie jest konieczne modelowanie gęstości danych ani

identyfikowanie szumu. Wystarczy określenie relacji sąsiedztwa na podstawie zadanej minimalnej odległości między pachołkami. Z tego względu zastosowanie DBSCAN uznano za niepotrzebnie rozbudowane w stosunku do wymagań zadania. Hierarchiczne grupowanie umożliwia budowanie hierarchii skupisk na podstawie odległości między obserwacjami lub grupami obserwacji. W tym przypadku nie ma jednak potrzeby tworzenia takiej hierarchii ani analizowania różnych poziomów grupowania. Wybór konkretnego wariantu grupowania hierarchicznego mógłby ponadto prowadzić do innych rezultatów niż oczekiwane w zadaniu. Ostatecznie wybrano podejście wykorzystujące Union-Find do łączenia odczytów spełniających warunek odległości oraz Spatial Hashing do ograniczenia liczby sprawdzanych par punktów.

```r
library(ggplot2)
library(gridExtra)
library(ggrepel) # Kluczowe do inteligentnego rozmieszczania etykiet

# --- 1. Bezpieczne wczytywanie danych oryginalnych ---
orig_lines <- readLines("data/DaneDoZadań/zad1/example_input.txt")
n_orig <- as.numeric(orig_lines[1])
eps <- as.numeric(orig_lines[2])

orig_list <- lapply(3:(2 + n_orig), function(i) {
  parts <- as.numeric(strsplit(orig_lines[i], "\\s+")[[1]])
  data.frame(id = parts[1], x = parts[2], y = parts[3])
})
original_data <- do.call(rbind, orig_list)

# --- 2. Bezpieczne wczytywanie plików wynikowych ---
read_output_custom <- function(filepath) {
  lines <- readLines(filepath)
  n <- as.numeric(lines[1])
  if (n == 0) return(data.frame(id=integer(), x=numeric(), y=numeric(), count=integer(), merged_str=cha

  res <- lapply(2:length(lines), function(i) {
    parts <- as.numeric(strsplit(lines[i], "\\s+")[[1]])
    id <- parts[1]
    x <- parts[2]
    y <- parts[3]
    count <- parts[4]
    orig_ids <- parts[5:length(parts)]

    data.frame(
      id = id,
      x = x,
      y = y,
      count = count,
      merged_str = paste0("[", paste(orig_ids, collapse = ","), "]"),
      stringsAsFactors = FALSE
    )
  })
  do.call(rbind, res)
}

greedy_data <- read_output_custom("output_greedy.txt")
spatial_data <- read_output_custom("output_spatial.txt")

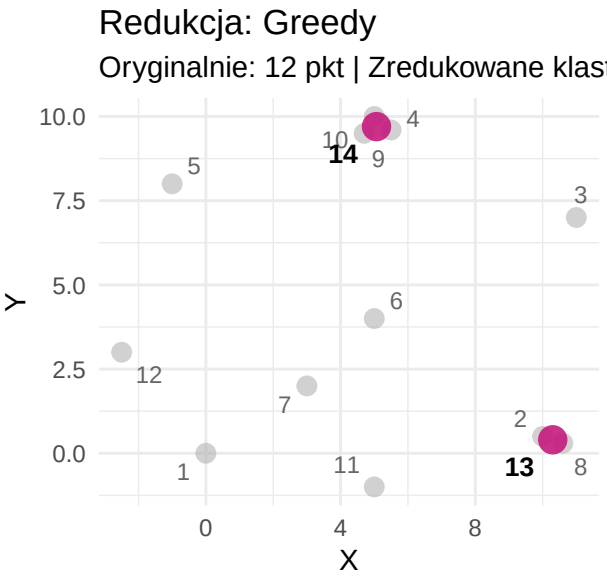
# --- 3. Funkcja rysująca z ggrepel ---
```

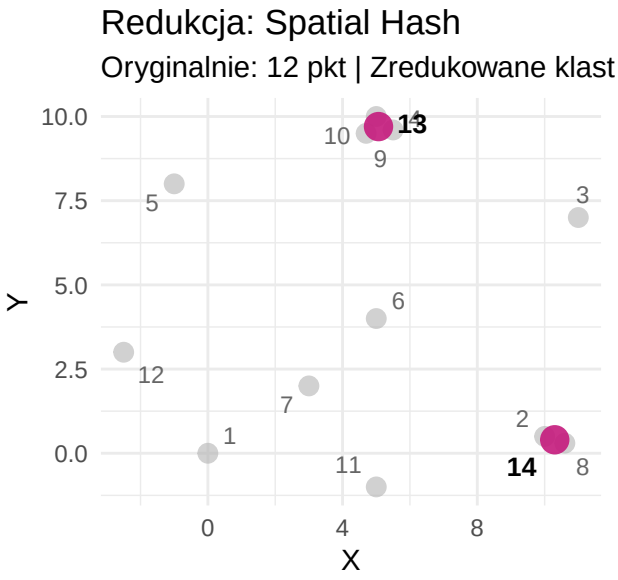
```r
plot_deduplication <- function(reduced_df, orig_df, title_text) {
  ggplot() +
    # Szare punkty oryginalne w tle
    geom_point(data = orig_df, aes(x = x, y = y), color = "grey70", size = 3, alpha = 0.6) +
    # Etykiety oryginalnych punktów (rozsuwane automatycznie)
    geom_text_repel(
      data = orig_df,
      aes(x = x, y = y, label = id),
      size = 3,
      color = "grey40",
      box.padding = 0.3,
      point.padding = 0.3
    ) +

    # Nowo powstałe, zredukowane klastry (różowe punkty)
    geom_point(
      data = reduced_df,
      aes(x = x, y = y),
      color = "#c51b7d",
      size = 4.5,
      alpha = 0.9
    ) +
    # Etykiety nowych klastrów (pogrubione, wyraźne, bez nakładania)
    geom_text_repel(
      data = reduced_df,
      aes(x = x, y = y, label = id),
      size = 3.5,
      fontface = "bold",
      color = "black",
      box.padding = 0.4,
      point.padding = 0.4
    ) +
    theme_minimal() +
    theme(legend.position = "none") +
    labs(
      title = title_text,
      subtitle = paste("Oryginalnie:", nrow(orig_df), "pkt | Zredukowane klastry:", nrow(reduced_df)),
      x = "X",
      y = "Y"
    ) +
    coord_fixed()
}

# Generowanie wykresów
p1 <- plot_deduplication(greedy_data, original_data, "Redukcja: Greedy")
p2 <- plot_deduplication(spatial_data, original_data, "Redukcja: Spatial Hash")

# Wyświetlenie obok siebie
grid.arrange(p1, p2, ncol = 2)
```





1.2

Zadanie 2

2.1

2.2